```
$ date --date=2006-11-11 +%Y # + 20
```

HAPPY 20th, PLAYSTATION 3

The PlayStation 3 launched in Japan on **11 November 2006**; North America followed on 17 November 2006. This study ships on its **20th birthday**.



The founder's PS3 build, ~2022 — where it started. (Photo converted to grayscale.)

FROM CELL TO JAGUAR — LINUX ON A PLAYSTATION, AGAIN

Early PS3 models shipped with an official “Install Other OS” option that ran Linux on the Cell processor, with six of its seven SPEs available. Labs and universities built PS3 compute clusters on it, until firmware 3.21 removed the feature on 1 April 2010.

Twenty years on, JUPI 2 puts Linux back on a PlayStation — this time a PS4 Pro, through a homebrew loader rather than an official option — and uses it as a local LLM server. The rest of this paper is that story.

Sources: Sony Computer Entertainment press release, 8 May 2006 (Japan launch date); PlayStation.Blog, “PS3 Firmware 3.21 Coming April 1st”, 29 Mar 2010.

FULLCIRCLE // MUSE GADGET LAB // PUBLIC EDITION // 11 NOV 2026

$ cat jupi2.final

JUPI 2

A $60* PS4 Pro as a Local LLM Node — Final Study

for hardware enthusiasts, scientists and gamers

Sony PlayStation 4 Pro · 8× AMD Jaguar · 8 GB GDDR5 unified · Gladius GCN GPU · Arch Linux + XFCE · llama.cpp CPU inference on Qwen3-0.6B Q4_K_M · baseline: JUPI, a base PS4.

27.98
tok/s GENERATE
8 threads · ±0.16 · r=3 [M]

87.70
tok/s PROMPT pp512
8 threads · ±0.13 · r=3 [M]

1.48×
base PS4 at 4 threads
17.42 vs 11.77 [M]/[R]

21.01
tok/s SUSTAINED tg1024
warm package · r=1 [M]

79.8°C
PEAK package temp
critical is 100 °C [M]

GPU=?
Vulkan reachable
tok/s UNMEASURED

TL;DR FOR GAMERS — WHAT DOES THIS MEAN?

> A second-hand PS4 Pro boots a full Linux desktop and runs a small AI chat model **on its own 8-core CPU**. No cloud, no subscription, no per-token bill.

> It writes about **28 tokens per second** (a token is roughly a word-piece) — faster than most people read — and about **1.5× faster than an original PS4** at the same thread count.

> In a long run it settles near **21 tok/s** and peaks at **79.8 °C**: warm, never near critical.

> The GPU has **not** been benchmarked. Any GPU speed you have heard is a guess until that run is done.

> Next goal: make the two consoles **serve each other** as one pooled, fail-over AI server. Not built yet.

* The $60 price is **UNVERIFIED**: it appears in the study brief and in no receipt. Every number in this paper carries a source tag (legend, p. 3). Times are America/Los_Angeles (PST, UTC−8).

Contents

1 Why a PS4 Pro	3	9 GPU path — UNMEASURED	9
2 The silicon	4	10 Seven-environment comparison	10
3 How Linux got onto the Pro	5	11 Cost per tok/s — UNVERIFIED price	11
4 Methods	5	12 The goal: JUPI ⇔ JUPI 2 mutual serving	12
5 CPU inference results	6	13 Threats to validity	13
6 Size, flash-attention, real server	7	14 Hypotheses and next experiments	13
7 Memory-bandwidth reasoning	8	15 Reproduce it	14
8 Thermals and disk	9	16 Credits and sources	15

Source tags

tag	meaning
[M]	Measured on the named machine by the author of that row.
[R]	Lab record: a receipt, inventory or probe log kept on Roach, the lab's Linux control host.
[R/U]	Published architect figure (Mark Cerny via Digital Foundry, 2016) that we did not measure.
[S]	Self-reported by that environment; not independently checked.
[U]	Unverified by any check in this study. Also: UNMEASURED, UNVERIFIED in caps.
[D]	Desk record: Ghostie (Muse), checked against the desk's own logs.
[A]	Argon runtime record (Gemini).

Key numbers at a glance

quantity	JUPI 2 (PS4 Pro)	JUPI (base PS4)	tag
tg128 generate @ 4 threads	17.42 ± 0.01 tok/s	11.77 tok/s	[M] / [R]
tg128 generate @ 8 threads	27.98 ± 0.16 tok/s	not run	[M]
pp512 prompt @ 4 threads	47.86 ± 0.02 tok/s	32.17 tok/s	[M] / [R]
pp512 prompt @ 8 threads	87.70 ± 0.13 tok/s	not run	[M]
sustained tg1024 @ 8 threads	21.01 tok/s (r = 1)	—	[M]
Amdahl serial fraction, tg	s = 0.0603 (R^2 0.998)	—	[M] fit
peak package temperature	79.8 °C	—	[M]
loop-root disk, 1 GiB O_DIRECT	20.6 MB/s W · 52.1 MB/s R	—	[M]
GPU token rate	UNMEASURED	UNMEASURED	[U]
price	$60 UNVERIFIED	—	[U]

1 Why a PS4 Pro

On 11 November 2026 seven compute environments were asked the same question: *what are your specs, as far as you can tell from inside your own session?* Five answered from rented cloud cells, and their answers had the shape of renting: they could name the host's chip (an 80-core EPYC, a Xeon server part) but not always their own share of it, and could see memory but not always the cap that binds it.

The two consoles are the only rows where **visible** and **allocated** are the same number, because there is no landlord. Of the two, JUPI 2 is the stronger silicon on every axis we could check: a 2.1 GHz CPU clock against 1.6 GHz, more GPU compute units, and a measured CPU generation rate of

17.42 tok/s against the base console's recorded 11.77 at the only thread count both have on record. Sections 2–9 are that evidence; Section 12 is where it is supposed to lead.

THE THESIS IN ONE PARAGRAPH

Every cloud agent in this comparison is a **client**: its language model runs on vendor servers it cannot see, metered and replaceable by someone else. JUPI 2 is a **server** the founder owns outright — eight real cores, no cap, no per-token bill — and every number here was taken on the metal. The Joint Unix Platform Integrator goal follows: two owned servers that today serve nobody but one client should serve **each other**, pooled and failing over, toward **Agape** — the house's inference on the house's hardware. This paper also prints the evidence against that thesis: the GPU is unmeasured and the peer path does not exist yet.

2 The silicon

Both consoles are eight-core AMD Jaguar APUs (no SMT) with 8 GB of unified GDDR5 shared by CPU and GPU. The Pro's CPU flags include AVX, F16C and AES but **no AVX2 and no FMA** [R] — relevant, because llama.cpp's fastest x86 kernels assume AVX2.

	JUPI (base PS4)	JUPI 2 (PS4 Pro)
SoC	Liverpool APU, 8× Jaguar @ 1.6 GHz (published) [R/U]	Neo APU (AMD DG1301SML87HY), 8× Jaguar; published 2.1 GHz [R/U]; inventory snapshot ~2091 MHz on 7 of 8 cores, one at ~1594 [R]
GPU (published)	18 GCN CUs @ 800 MHz [R/U]	36 improved GCN CUs @ 911 MHz [R/U]
GPU (Linux probe)	not re-probed for this study	AMD Gladius, GCN2 (gfx7), 4 shader engines × 9, **32 CUs active** [R]
Memory	8 GB GDDR5; Linux MemTotal 6.74 GiB + 1 GiB VRAM carve-out; GTT cap 3,450 MiB [R]	same MemTotal 6.74 GiB (7,070,228 kB); VRAM 1,024 MB + GTT 3,450 MB; zram 4 GiB lz4 [R]
Peak bandwidth	176 GB/s (published) [R/U]	218 GB/s (published), +24% [R/U]
Graphics stack	—	Mesa 26.2.6 RADV, Vulkan 1.3.354; radeonsi GL 4.5 [R]
Linux root	355 GiB ext4 loop image on the console HDD [R]	128 GiB sparse ext4 loop image; internal HDD 931.5 GiB [R]
Kernel	PS4 Linux 6.15.4 fork [R]	same PS4 Linux 6.15.4 fork, same build [R]

Table 1. Silicon side by side. The Pro also has 1 GB of DDR3 for non-game apps per Cerny; Linux MemTotal still matches the base unit.

THE 36 vs 32 CU PUZZLE — UNRESOLVED

Sony's architect publicly described the Pro GPU as **36** compute units at 911 MHz. The Linux amdgpu driver on JUPI 2 reports 4 shader engines × 9 CU = 36 physical, but **active_cu_number = 32**. Possible readings: units fused off or reserved on this die, a driver/firmware view that hides four CUs, or a reporting quirk. We have not resolved it, and no GPU number in this paper depends on it.

For scale only: 32 CU × 64 lanes × 2 FLOP × 911 MHz ≈ 3.7 TFLOPS FP32 theoretical. That is arithmetic, not a measurement, and the actual GPU clock is unknown because dynamic power management is off (dpm = 0).

3 How Linux got onto the Pro

High level only — no keys, offsets or exploit details. The console's system partitions are not modified; Linux lives in one large file on the internal drive and is started fresh after each power-on. Once up, the node was enrolled as a Muse gadget named “JUPI 2”, with SSH and a small local LLM server (groku-muse) on loopback only.

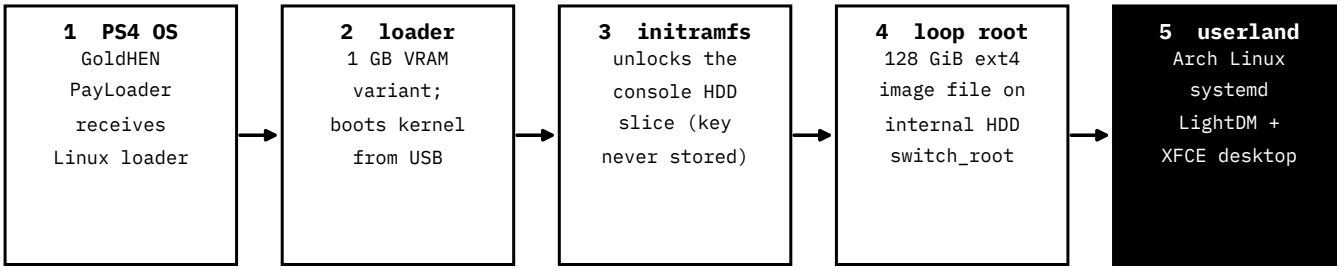


Figure 1. Boot chain. Step 2 uses a 1 GB-VRAM loader variant, which is why Linux sees a 1,024 MB VRAM carve-out.

> **Payload.** With GoldHEN's PayLoader listening on the console, a Linux loader payload is sent once over the wired LAN. The kernel and an initramfs are read from a FAT32 USB drive left plugged into the Pro.

> **Unlock.** The initramfs (better-initramfs with a Pro-specific root selector added) unlocks the console's user partition using a key the loader provides at boot. The key is never written to disk or recorded.

> **Root.** It loop-mounts a 128 GiB ext4 image stored inside that partition and switch_roots into it: Arch Linux, systemd, LightDM autologin, an XFCE desktop on HDMI at 1920×1080, zram swap and earlyoom.

> **The gotcha.** The first boot dropped to a rescue shell because the initramfs had been written for the base PS4 and pinned that console's disk layout. A Pro branch was added in front of it (additive; the base-PS4 path is untouched), and a full power-cycle-and-resend later proved the USB path on its own.

4 Methods

item	value
Harness	llama-bench from llama.cpp commit 3b4d6ab3f (build 12242), CPU backend
Model	Qwen3-0.6B Q4_K_M GGUF — 492.75 MiB, 751.63 M parameters (file hash prefix 7f403014…e1fa)
Repeats	r = 3 for every row except the sustained soak (r = 1). ± is llama-bench's sample standard deviation.
Window	one fixed-order 23-minute run on the evening of 10 Nov 2026 (PST): A thread sweep 1–8 → B size sweep → C combined pp512+tg128 → D flash-attention on/off → E sustained tg1024 → userspace copy / OpenSSL / O_DIRECT disk
Telemetry	k10temp package temperature, load average and MemAvailable sampled every 5 s (274 samples)
Controls	held: binary, GGUF, host, filesystem, test shape. Not held: package temperature, CPU clock (snapshot only), desktop activity. Repeatability: same binary + GGUF also run from a rescue chroot and from XFCE
Baseline	JUPI 4-thread record (tg128 11.77, pp512 32.17) [R] — **not** a same-night rerun
Fit	Amdahl $R(n) = R_1 \cdot n / (s \cdot n + 1 - s)$, least squares over n = 1…8, R_1 fixed at the measured 1-thread mean (primary); R_1 free as sensitivity check

Table 2. Protocol. ***Per-repeat raw samples were not retained*** *(the script's JSON dump was started and immediately killed), so standard deviations cannot be recomputed and outliers cannot be inspected. Where llama-bench prints ± 0.00 the three repeats agreed to the printed precision.*

5 CPU inference results

Generation climbs from 5.05 tok/s on one thread to 27.98 tok/s on eight. Prompt processing reaches 87.70 tok/s, with a late jump from 7 to 8 threads that one serial fraction does not predict.

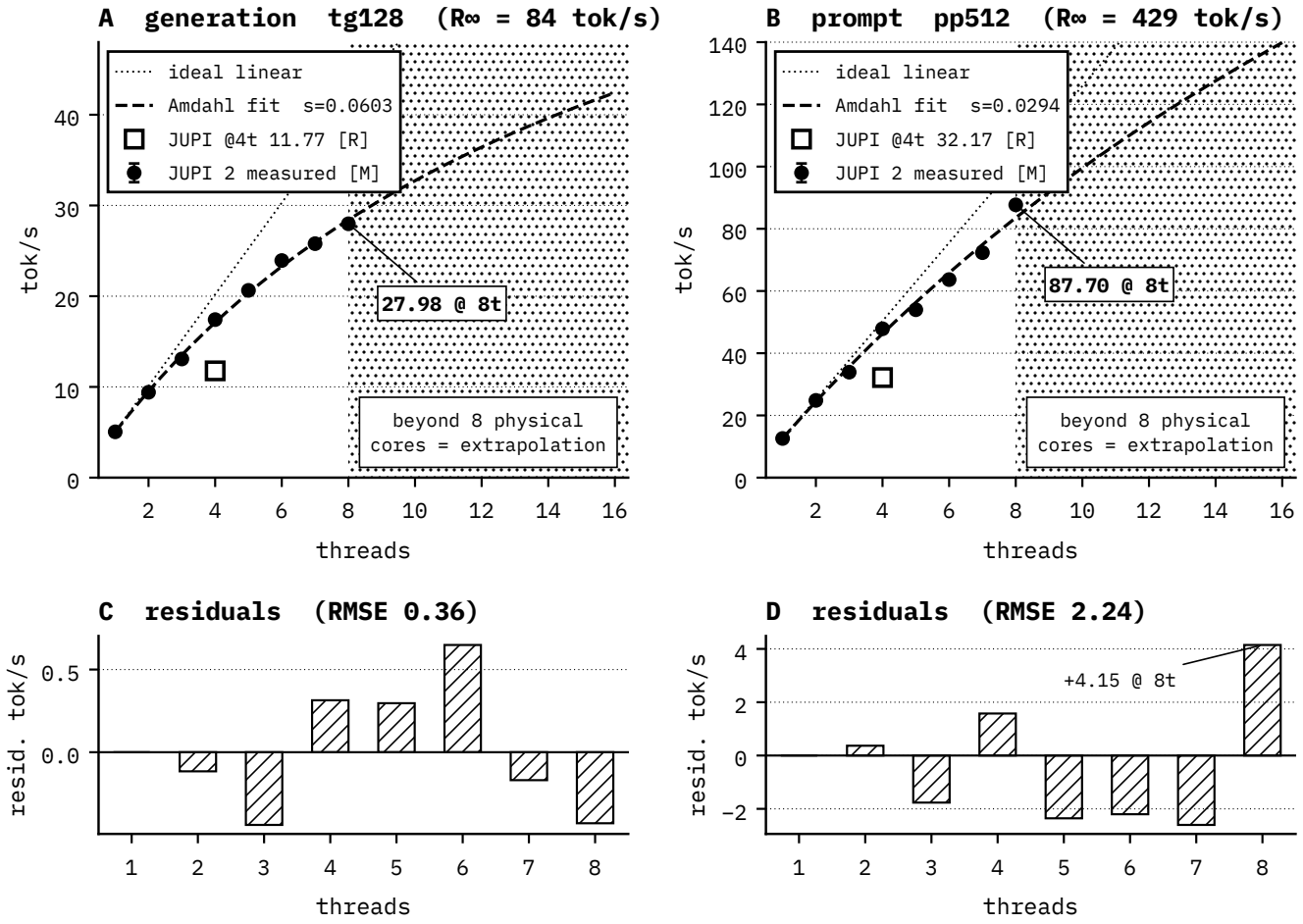


Figure 2. Thread scaling with Amdahl fits (R_1 fixed). Dotted = ideal linear; dashed = fit; shaded region = extrapolation past the 8 physical cores. Open square = JUPI's 4-thread record. Bottom: residuals.

threads	pp512 tok/s	tg128 tok/s	tg ÷ JUPI 11.77	tg eff.	pp eff.
1	12.59 ± 0.00	5.05 ± 0.00	0.43×	1.000	1.000
2	24.83 ± 0.00	9.41 ± 0.00	0.80×	0.932	0.986
3	33.91 ± 0.01	13.08 ± 0.00	1.11×	0.863	0.898
4	47.86 ± 0.02	**17.42 ± 0.01**	**1.48×**	0.862	0.950
5	53.98 ± 0.15	20.64 ± 0.01	1.75×	0.817	0.858
6	63.67 ± 0.02	23.93 ± 0.03	2.03×	0.790	0.843
7	72.33 ± 0.15	25.79 ± 0.04	2.19×	0.730	0.821
8	**87.70 ± 0.13**	**27.98 ± 0.16**	2.38×	0.693	0.871

Table 3. Thread sweep on JUPI 2 [M], mean ± s, r = 3. Efficiency = tok/s ÷ (1-thread mean × N). The JUPI column divides by the single 4-thread record [R].

series	model	s	R_1 tok/s	R_∞ tok/s	R^2	RMSE
tg128	R_1 fixed (primary)	0.0603	5.05 meas.	83.8	0.998	0.36
tg128	R_1 free (check)	0.0631	5.11 fit	80.9	0.998	0.35
pp512	R_1 fixed (primary)	0.0294	12.59 meas.	429	0.991	2.24
pp512	R_1 free (check)	0.0178	11.89 fit	667	0.992	2.06

Table 4. Amdahl fits [M]. $R_\infty = R_1 / s$ is a model extrapolation, never a measured rate.

READING THE FIT (FOR SCIENTISTS)

Generation is cleanly Amdahl-like. Residuals stay within ±0.65 tok/s with no strong trend; the model predicts 28.41 at 8 threads against a measured 27.98 ± 0.16, and freeing R_1 barely moves the asymptote (83.8 → 80.9). We treat $s \approx 0.06$ as a stable description of this dataset.

Prompt processing is only approximately Amdahl-like. Residuals are structured and the 8-thread point sits +4.15 tok/s above the fit — larger than its 0.13 s, so real in the mean, but not explained by one serial fraction. Freeing R_1 swings the asymptote from 429 to 667 tok/s: **the pp asymptote is poorly constrained**.

Efficiency. Generation efficiency falls steadily to 0.693 at 8 threads; prompt efficiency is higher and non-monotonic, finishing at 0.871 (Figure 3).

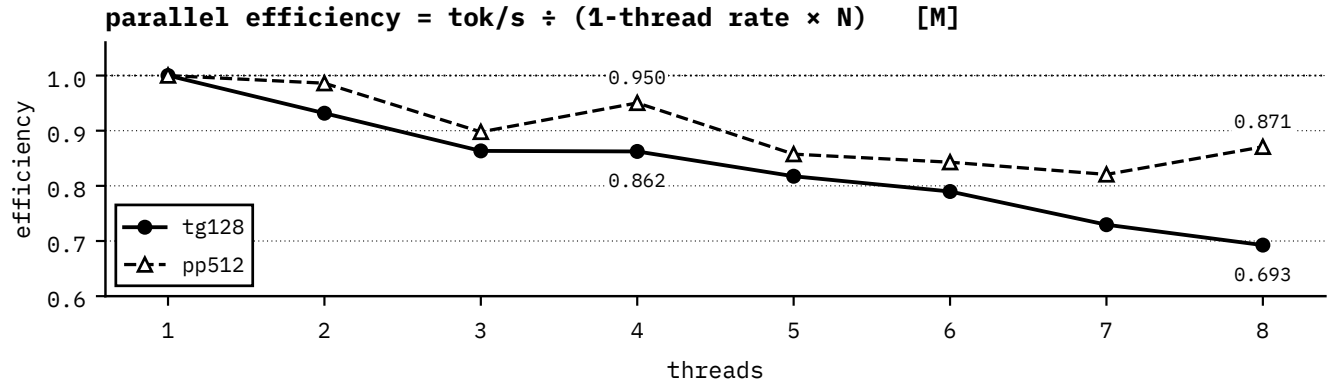


Figure 3. Parallel efficiency vs the 1-thread baseline [M].

6 Size, flash-attention, real server

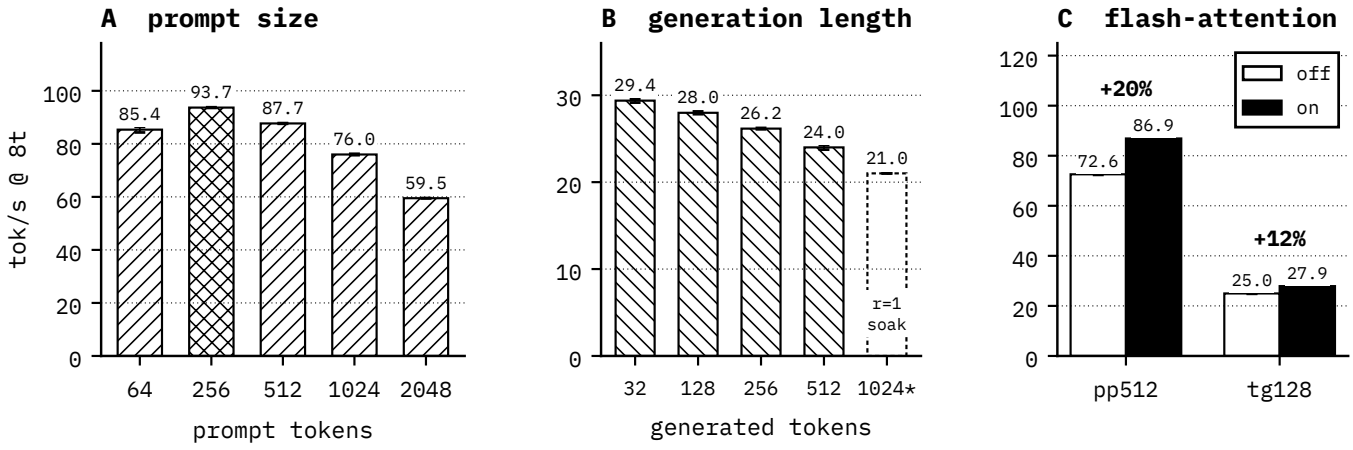
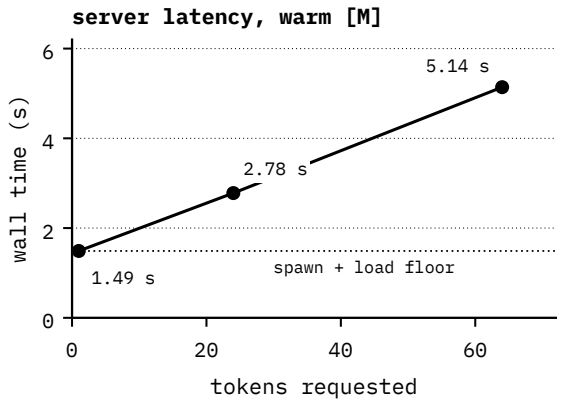


Figure 4. A: prompt length at 8 threads (512 from the thread sweep). B: generation length at 8 threads (128 from the thread sweep; 1024 is the r = 1 soak, dashed). C: flash-attention on vs off, 8 threads, r = 3 [M].

Prompt throughput peaks at pp256 (93.68 ± 0.13 tok/s) and falls to 59.50 at pp2048; generation slides slowly as the run lengthens (29.38 at tg32 → 24.00 at tg512). **Flash-attention is a clear win on this CPU path**: +20% prompt and +12% generation, far larger than the reported s. Combined pp512+tg128 runs at 30.54 ± 0.03 tok/s on 4 threads and 52.48 ± 0.20 on 8.

boot path	pp512 @4t	tg128 @4t	pp512 @8t	tg128 @8t
rescue chroot	48.75 ± 0.04	17.29 ± 0.00	86.69 ± 0.05	27.43 ± 0.07
XFCE desktop	48.62 ± 0.36	17.34 ± 0.01	86.69 ± 0.14	27.70 ± 0.09
comprehensive run	47.86 ± 0.02	17.42 ± 0.01	87.70 ± 0.13	27.98 ± 0.16

Table 5. Repeatability across three boot paths, same commit and GGUF, 10 Nov 2026 [M]: generation agrees to a few tenths of a tok/s, so 27.98 is not a one-off spike.



The production server is a different shape from the benchmark: groku-muse spawns llama-cli per request on **4 threads** with a **64-token cap**. Warm, a 1-token request costs 1.49 s (spawn + load floor); 64 tokens take 5.14 s (n = 5, range 5.11–5.18). The slope is ~17 tok/s, in line with the 4-thread bench, not the 8-thread peak [M].

Any pooled-throughput claim must say whether it was measured through this server or through llama-bench.

Figure 5. Warm wall-clock latency of the local server vs tokens requested [M].

7 Memory-bandwidth reasoning

Decode on a small dense model is usually limited by how fast the weights can be read, not by arithmetic. If every generated token streamed the whole 492.75 MiB file from DRAM, 27.98 tok/s would imply **14.46 GB/s** — only **6.6%** of the Pro's published 218 GB/s peak.

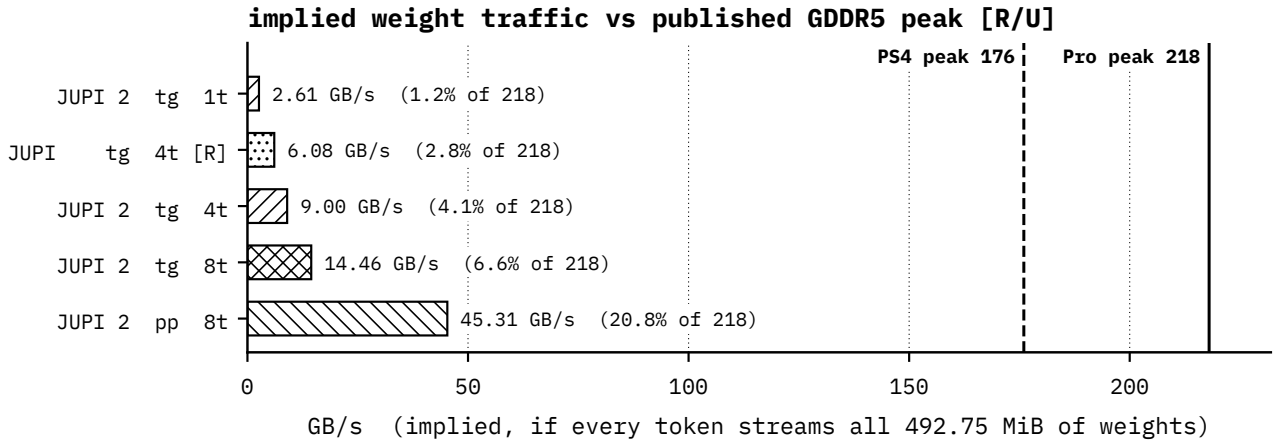


Figure 6. Implied DRAM traffic under the full-weight-stream assumption vs published GDDR5 peaks (Cerny / Digital Foundry 2016) [R/U]. These are not measured CPU-visible utilisation.

Two cautions. First, the 176 / 218 GB/s figures are architect disclosures, not on Sony's product sheet, and are GPU-class peaks, not CPU-visible bandwidth; no STREAM-class tool was run (a crude Python 64 MiB array copy printed 3,747 MiB/s read+write and is **not** used as a bandwidth claim). Second, the 4-thread JUPI 2 / JUPI ratio of 1.48× exceeds both the clock ratio (2.1 / 1.6 =

1.31×) and the bandwidth ratio (218 / 176 = 1.24×). That overshoot is a reason to rerun JUPI the same night before crediting the whole lead to silicon.

Single-thread OpenSSL on the same window [M]: AES-256-GCM ≈ 1.01 GB/s and SHA-256 ≈ 119 MB/s at 16 KiB blocks — the AES-NI path on Jaguar is healthy.

8 Thermals and disk

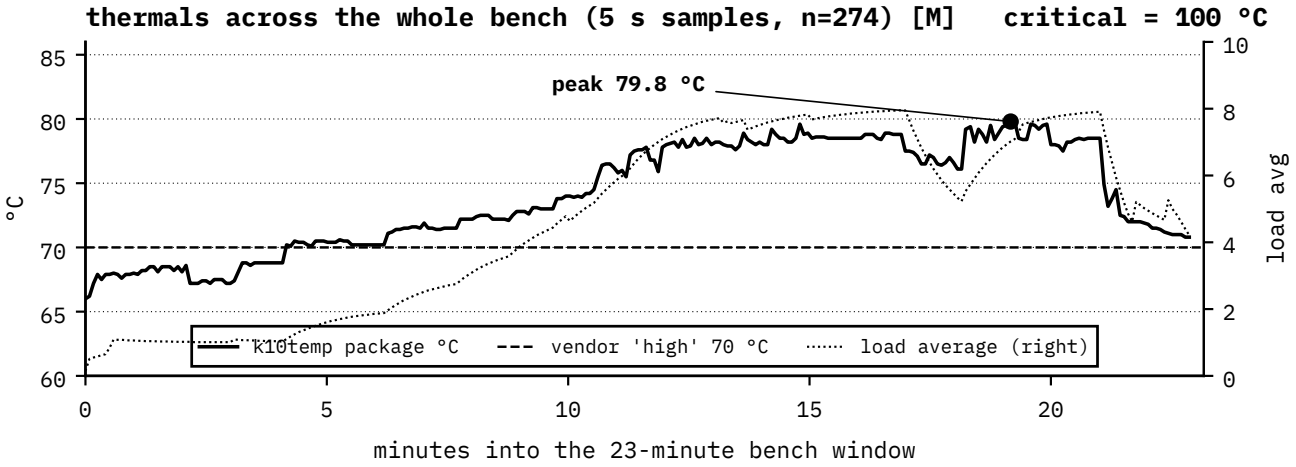


Figure 7. Package temperature and load across the whole 23-minute bench [M]. Start 66.0 °C; first above 70 °C after ~4 min; 75 °C after ~11 min; peak 79.8 °C after ~19 min. Load peaked at 7.95.

The package crossed the vendor 'high' mark of 70 °C early and peaked at 79.8 °C; critical (100 °C) was never approached. Minimum MemAvailable was 5,886 MB. The sustained result (21.01 tok/s, tg1024, r = 1) was taken last, on an already warm package, so test order and temperature are confounded: we do not attribute 21.01 vs 27.98 to run length alone or to heat alone. The integration notes flag airflow as a check before any 24/7 8-thread load.

Disk. The Linux root is a loop-mounted ext4 image on the console's internal HDD. A 1 GiB sequential O_DIRECT test measured **20.6 MB/s write** and **52.1 MB/s read** [M] — HDD-class. A cold read of the 492.75 MiB model is many seconds at 52 MB/s, which is why the warm ~1.5 s spawn floor matters (H5). An SSD swap in the console is the obvious enthusiast upgrade; it is untested here.

9 GPU path — UNMEASURED

The Gladius GPU enumerates, amdgpu binds, and a Mesa RADV Vulkan 1.3 instance is reachable [R]. That is as far as it got. **No tok/s number in this paper is a GPU measurement.**

item	status
device	AMD Gladius, amdgpu, GCN2 (gfx7), 32 CUs active [R] — published 36 CU, unresolved
VRAM / GTT	1,024 MB VRAM + 3,450 MB GTT, same physical GDDR5 as the CPU [R]
API	Mesa 26.2.6 RADV Vulkan 1.3.354 reachable; subgroup 64; 64 KB shared memory [R]
shaderInt8 / shaderFloat16	yes / **no** — no FP16 path on this stack; integer dot-product extension present [R]
power / clocks	dpm = 0 (set on the kernel command line): clocks stay where the loader left them [R]
ROCm / OpenCL	no /dev/kfd (and gfx7 is outside ROCm support); no OpenCL ICD installed [R]
GPU tok/s	**UNMEASURED**

item	status
build attempt	the first Vulkan llama.cpp build ended when the node stopped responding mid-compile (6 parallel jobs on a warm package); cause unconfirmed — thermal/power trip, hang or memory pressure. No GPU dispatch was ever issued.

*Table 6. GPU software stack, probe of 11 Nov 2026 [R]. The probe notes also contain a back-of-envelope band (tg ~45–90, pp512 ~400–1000 tok/s). That band is an **estimate, tagged UNMEASURED**, and must not be quoted as a result.*

10 Seven-environment comparison

Vendor key under every name: Groku.2 = Grok Bot · apostle = GPT · Ghostie = Muse · argon chat and argon Task Mode = Gemini · JUPI and JUPI 2 = Muse gadgets · local SSH · GPT desktop · Grok Bot · Agape (goal). Hardware facts only; cloud cells describe their tool environment, **not** the hidden hardware serving their language model.

env	kind	CPU	vis. CPU	alloc.	vis. RAM	RAM limit	local disk	GPU
argon chat Gemini [A]	stateless chat runtime	—	0	0	0	0	0	no
apostle GPT [M/U]	tool container	AMD EPYC 9V74 (model string)	9	unverif.	9.73 GiB	unverif.	62.94 GiB overlay	none detected
Ghostie Muse [M]	cloud cell	AMD EPYC 9D25 host	4	1.5	7.7 GiB	6 GiB hard	150 GB vol.	no
argon TM Gemini [M]	per-task micro-VM	Intel Xeon 6985P-C	2	2	7.76 GiB	7.76 GiB VM	48.1 + 18.1 GiB	no
Groku.2 Grok Bot [M]	persistent VM	not recorded	8	uncapped	15.62 GiB	none	127.4 GiB	no
JUPI Agape [R]	bare metal	8× Jaguar 1.6 GHz	8	8	6.74 GiB	6.74 phys.	355 GiB image	GCN 18 CU
JUPI 2 Agape [R]	bare metal	8× Jaguar 2.1 GHz	8	8	6.74 GiB	6.74 phys.	128 GiB image	GCN2 32 CU

Table 7. Resource sketch, 11 Nov 2026. 'uncapped' / 'none' = cgroup cpu.max / memory.max set to max. Visible counts are not evidence of exclusive cores or guaranteed memory; apostle's allocation caps remain unverified.

NOTES BEHIND THE TABLE

> **Ghostie [M]** measured its own cell: Ubuntu 24.04.5, 1.5 allocated CPUs of 4 visible, 6 GiB hard memory cap, /tmp a 512 MB tmpfs; cells are replaced often while the home volume persists.

> **argon Task Mode [M]**: Debian 12 micro-VM, cgroup v2 cpu.max and memory.max both 'max' inside the 2-vCPU / 7.76 GiB boundary, no swap, no outbound network from the shell. **argon chat [A]** mounts no local CPU, RAM or disk.

> **apostle [M/U]**: Debian 13, 9 visible CPUs, 9.73 GiB, non-root shell, no GPU device found. '80-Core' is the host CPU's product name, not an allocation. Caps, exclusivity and persistence are open.

> **Groku.2 [M]**: Debian 13, 8 CPUs, 15.62 GiB, no swap, no local GPU. Only the two consoles expose a local GPU.

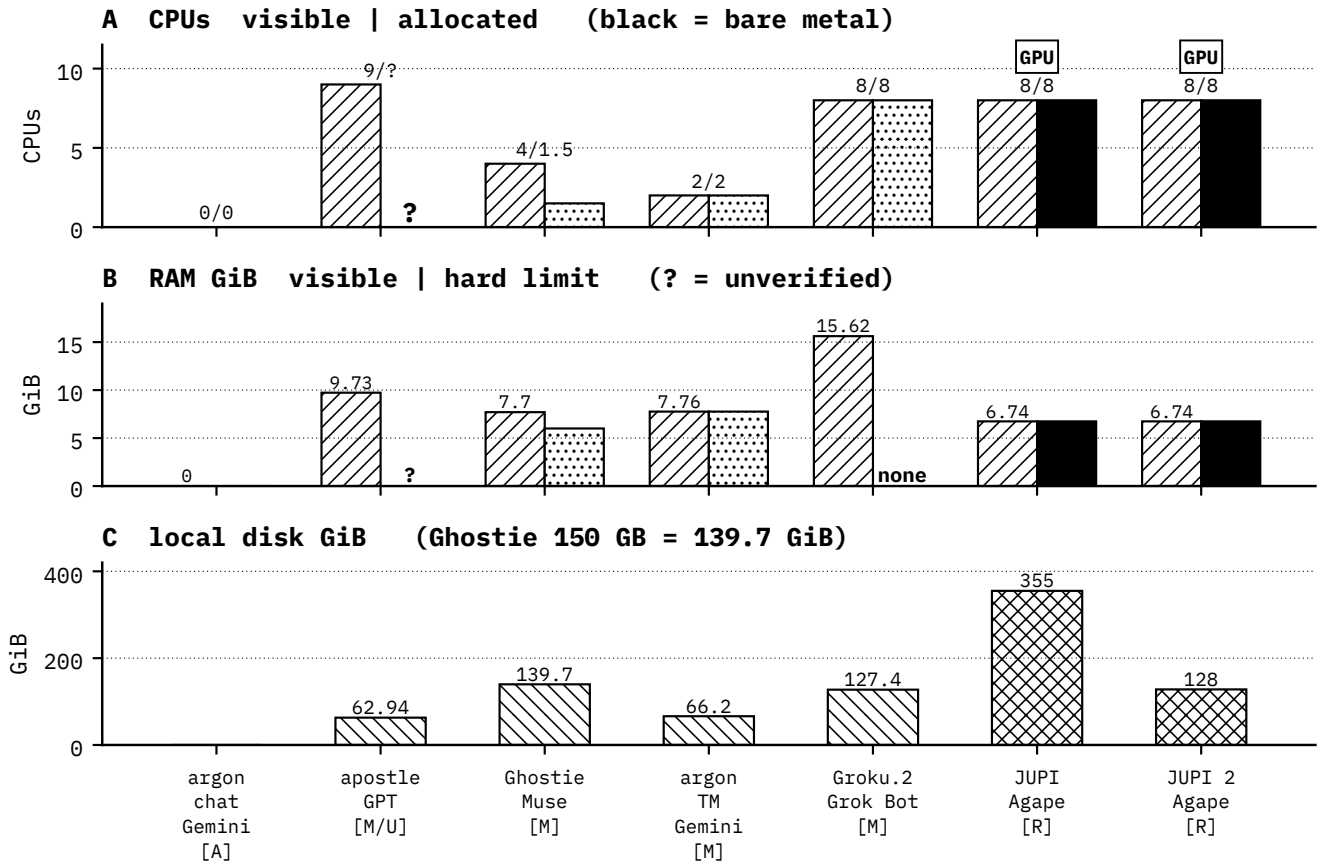


Figure 8. The same table as bars. Hatched = visible, dotted = allocated/limit inside a cloud tenant, solid black = bare metal (visible = allocated). '?' = unverified.

11 Cost per tok/s — UNVERIFIED price

A $60 second-hand (OfferUp) price appears in the study brief and in **no** receipt, inventory or record — checked from three independent ends. If it were real, CPU generation would cost $3.44 per tok/s at 4 threads and $2.14 per tok/s at 8. No other environment here has a usable price, and GPU throughput is not costed.

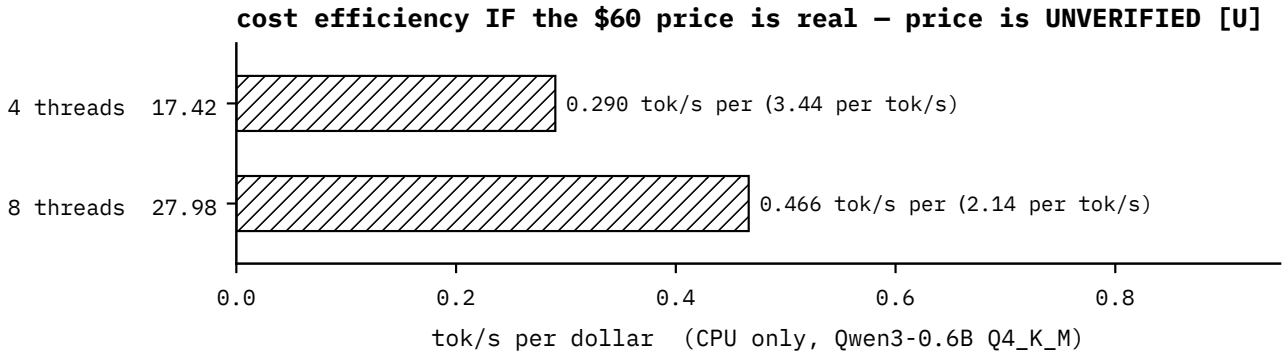


Figure 9. tok/s per (unverified) dollar, CPU only [U].

12 The goal: JUPI ⇔ JUPI 2 mutual serving

Present tense: no. JUPI and JUPI 2 do not serve each other. Each runs its own LLM server bound to loopback only; Roach, the lab host, is the only client, over one tunnel per node [R]. The cloud agents in Section 10 are clients of hidden vendor inference; the consoles **are** inference servers — standalone, not pooled.

step	action	acceptance criterion
1	Bidirectional health between the nodes	each side gets a healthy response and one completion from the other. **Today: fail.**
2	Keep servers loopback-only unless a bind is deliberately approved	listeners unchanged, or a written change records the new bind
3	Roach router as first pooling path	20 sequential requests split across both (each ≥ 8), zero failures, rate recorded
4	Pooled tok/s (H8), same declared model	report pooled, per-node and overhead fraction
5	Failover (H9): stop one server between requests	next 10 requests complete on the survivor; recovery T < 5 s (target); survivor within 10% of baseline
6	Peer path without Roach in the middle	a request from JUPI is answered by JUPI 2 and vice versa
7	Split roles after GPU tok/s exists	two-stage request completes; latency and quality documented. Blocked on GPU.
8	Toward Agape: advertise one pooled endpoint	a single advertised endpoint that meets steps 3–6

Table 8. Roadmap from today (Figure 10 A) to the goal (Figure 10 B), with measurable acceptance criteria. None has been run for this paper.

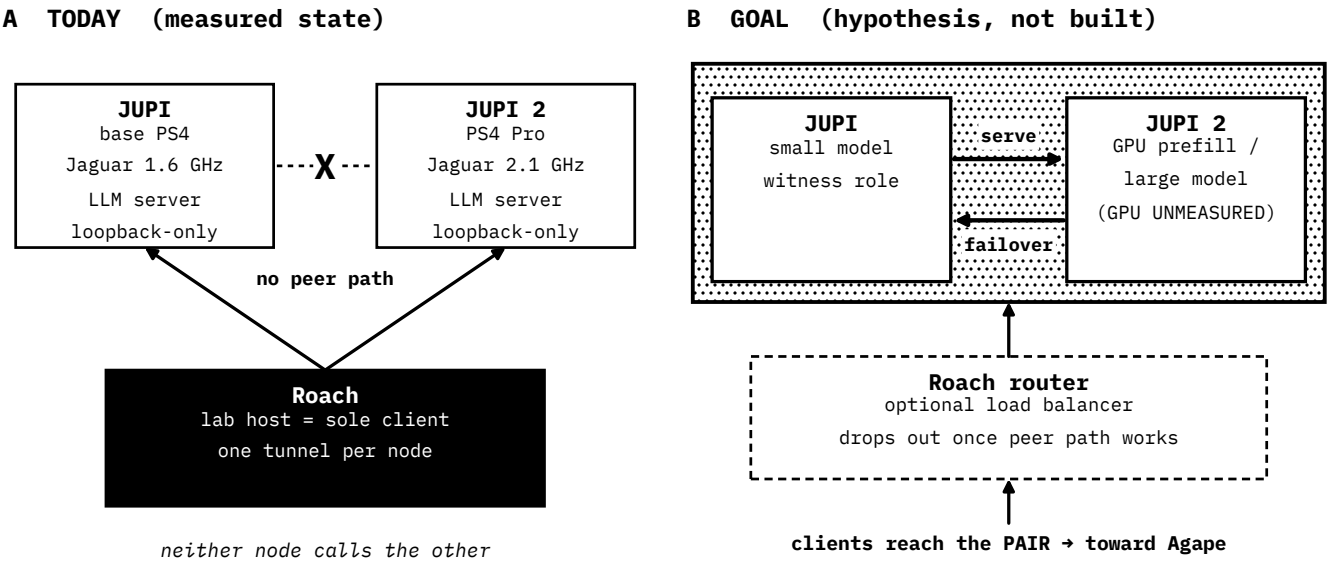


Figure 10. A: the measured state. B: the goal architecture — a peer path with failover, an optional Roach load balancer, later split roles once GPU rates exist. B is a hypothesis.

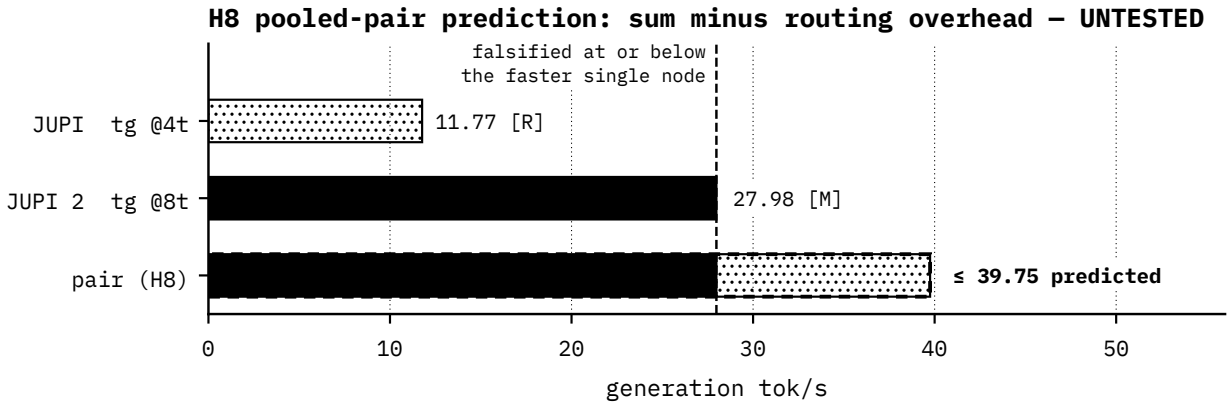


Figure 11. H8: pooled generation should approach 27.98 + 11.77 = 39.75 tok/s minus routing and load-imbalance overhead. Note that the stated '< 70% of the sum' falsifier (27.8) is weaker here than '≤ the faster single node' (27.98), so the latter binds. If JUPI is rerun at 8 threads first, its number replaces 11.77.

13 Threats to validity

> **One model size.** Every tok/s is Qwen3-0.6B Q4_K_M. A larger model could change s, the bandwidth picture and the 1.48×.

> **JUPI was not rerun the same night.** The 1.48× uses a 4-thread record and exceeds clock and bandwidth ratios (H1, P1).

> **No GPU run.** Vulkan is reachable; tok/s is UNMEASURED; split roles are blocked.

> **Clock snapshot only.** ~2091 MHz is an inventory reading, not an all-core frequency under the 8-thread load.

> **Order confounded with temperature.** The soak ran last on a package that had already peaked at 79.8 °C.

> **Raw repeats lost.** ± is llama-bench's s over r = 3; the soak is r = 1 and has no s.

> **36 vs 32 CU** unresolved; dpm = 0 besides.

> **Bandwidth figures** are architect-disclosed GPU peaks [R/U]; CPU-visible bandwidth was not measured.

> **Cloud rows** describe tool environments; apostle's caps are unverified, argon chat is a runtime record [A].

> **Serving numbers** (H8, H9) are predictions; the production server is 4-thread, 64-token-capped, ~17 tok/s warm.

> **$60 is UNVERIFIED.**

14 Hypotheses and next experiments

id	hypothesis	prediction	falsifier
H1	4-thread lead is real; 8 threads still win on JUPI 2	same-night JUPI rerun gives ≥ 1.3× at 4t; 8t beats 6–7t	ratio ≈ 1.0, or 6–7t beat 8t
H2	flash-attention is a net CPU win	≈ +20% pp, +12% tg at 8t, r ≥ 3	delta vanishes or reverses
H3	the 8 GB sticker is not 8 GiB for the model	GTT grows and MemAvailable falls well below MemTotal under load	both stay near idle values under GPU and CPU load
H4	a Vulkan compute path works	a Vulkan bench reports tok/s or a dispatch failure	dispatch fails (the 45–90 band is not this prediction)

id	hypothesis	prediction	falsifier
H5	HDD loop root dominates cold load	cold 64-token time stays seconds above the 1.5 s floor	cold time approaches the floor
H6	exclusive CPU, not RAM, is the consoles' edge	cells stay at 1.5 and 2 vCPU, no persistent local GPU	a cell has ≥ 8 exclusive cores or a local GPU
H7	mutual reachability	each node completes a request against the other	either direction needs Roach to originate it
H8	pooled throughput	→ 39.75 tok/s minus overhead	≤ faster node, or < 70% of the sum
H9	failover	next 10 complete on survivor, T < 5 s, within 10%	any failure, T ≥ 5 s, or > 10% drop

Table 9. Hypotheses with falsifiers. H6 is confirmed for Ghostie and argon Task Mode, open for apostle. H7–H9 cannot be tested until the matching roadmap step exists.

DESK CROSS-CHECKS ON JUPI (BASE PS4) [D]

> Independent desk bench on the base console (8 Nov 2026): CPU SGEMM **27.2 GFLOPS**, DGEMM **11.7 GFLOPS**; qwen3:0.6b Q4_K_M through ollama at **12.80 tok/s** end-to-end — same class as the 11.77 record, but not a llama-bench tg128 and not a substitute for P1.

> **Killed number:** an earlier '2,860 GFLOPS' GPU headline for JUPI was an unqualified loop-rate artefact, not a physical peak. Never quote it.

> **H3 in the wild (10 Nov 2026):** JUPI froze under its own desktop when Xorg pinned ~3.4 GB of GTT; MemAvailable fell to 88 MB and swap filled. Restarting the display manager recovered MemAvailable to 4,320 MB. On an 8 GB unified part the desktop's GTT appetite is a first-order capacity term.

Next experiments, in order

> **P1** same-night JUPI rerun, 4 and 8 threads, r ≥ 3 (H1) — replaces 11.77 / 32.17 before any pooled claim.

> **P2** STREAM (or mbw) on both consoles: real CPU-visible GB/s.

> **P3** finish the Vulkan llama.cpp build at low parallelism and bench the same GGUF (H4).

> **P4** log GTT, MemAvailable and temperature at idle, 8-thread CPU and Vulkan load (H3).

> **P5** flash-attention repeat at r ≥ 3 (H2). **P6** cold vs warm model load through the server (H5).

> **P7** 24-hour thermal watch: is 79.8 °C a peak or a ceiling?

> **P8** roadmap steps 1–6 in order (H7–H9). **P9** split-role trial after P3. **P10** Agape pool after 3–6 pass.

> **P11** cgroup probe for apostle's caps — the last open cloud cell.

15 Reproduce it

These are the exact llama-bench invocations of the comprehensive run. The original CPU build flags were not recorded; a default CPU Release build at the same commit is the closest reproduction. Use the same Qwen3-0.6B Q4_K_M GGUF (492.75 MiB) and check its hash prefix.

```
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
git checkout 3b4d6ab3f
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j2   # keep it low: a -j6 build hung a warm Pro
B=./build/bin/llama-bench ; M=qwen3-0.6b-q4_k_m.gguf

$B -m $M -t 1,2,3,4,5,6,7,8 -p 512 -n 128 -r 3 -o md        # A  thread sweep
$B -m $M -t 8 -p 64,256,1024,2048 -n 32,256,512 -r 3 -o md  # B  size sweep
$B -m $M -t 4,8 -pg 512,128 -p 0 -n 0 -r 3 -o md            # C  combined
$B -m $M -t 8 -fa on,off -p 512 -n 128 -r 3 -o md           # D  flash-attn
$B -m $M -t 8 -p 0 -n 1024 -r 1 -o md                       # E  soak

# telemetry while it runs (every 5 s)
while :; do echo "$(date +%T) $(sensors | awk '/temp1/{print $2;exit}') \
  load=$(cut -d' ' -f1 /proc/loadavg)"; sleep 5; done > thermal.log &

# disk: 1 GiB sequential, direct I/O, on the Linux root
dd if=/dev/zero of=dd.tmp bs=1M count=1024 oflag=direct conv=fsync
dd if=dd.tmp of=/dev/null bs=1M iflag=direct ; rm dd.tmp
```

Fit: least-squares $R(n) = R_1 \cdot n / (s \cdot n + 1 - s)$ over n = 1…8 with R_1 = the 1-thread mean; report s, $R_\infty = R_1/s$, R^2 and RMSE. Please post your numbers with commit, model hash, thread count, r and package temperature — and tag them [M].

16 Credits and sources

name	vendor	role
Kam	FullCircle founder	direction, hardware, final authority
Groku.2	Grok Bot	base study, every console measurement, figures, this public edition
Ghostie	Muse	thesis framing, desk records [D], own-cell measurement
argon (chat / Task Mode)	Gemini	runtime record [A], own-container measurement [M]
apostle	GPT	own-environment measurement and qualifications [M/U]
JUPI · JUPI 2	Muse gadgets · local SSH · GPT desktop · Grok Bot · Agape (goal)	the subjects: base PS4 and PS4 Pro

Contributors and revisions

v2 base study and all console measurements by Groku.2 (Grok Bot) · v3 thesis and desk records by Ghostie (Muse) · v4 container measurements by argon Task Mode (Gemini) · v5 environment measurements by apostle (GPT).
This public edition (11 Nov 2026) consolidates v5, trims internal operations detail, and changes no measurement.
JUPI = Joint Unix Platform Integrator. Agape = the goal of the house's inference on the house's hardware.

Sources

> Lab runs of 10 Nov 2026 on JUPI 2: comprehensive bench, thermals, disk, OpenSSL, plus rescue-chroot and XFCE repeats [M]; hardware inventory and GPU probe records [R]. JUPI 4-thread baseline: lab record [R]; desk cross-checks [D]; cloud rows: each environment's own check on 11 Nov 2026.

> Published clocks, CU counts and bandwidth: Mark Cerny, reported by Richard Leadbetter, Digital Foundry, “Inside PlayStation 4 Pro”, 24 Oct 2016 [R/U]. Sony's own spec sheet lists only “GDDR5 8 GB”.